

Extending DiskANN



Towards Adaptive and Self-Learning ANN Systems

Sai Jagadeesh · Krish Dange · Abhijeet Kumar

Team 3 | Making DiskANN Adaptive

Why Nearest Neighbor Search Matters



Applications

- Recommendation systems
- Search engines
- AI embeddings & retrieval



The Challenge

- High-dimensional spaces
- Curse of dimensionality
- Exact search = $O(n)$ scan

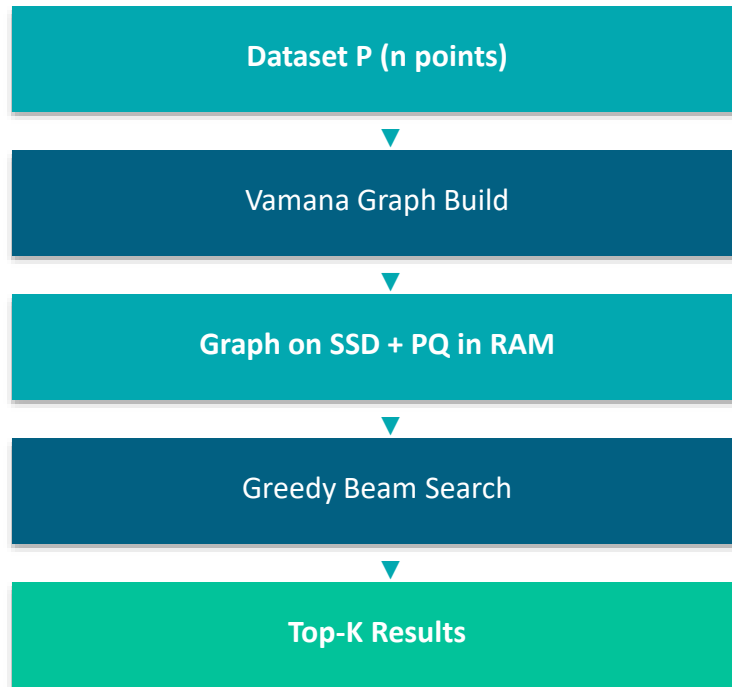


Why It's Hard

- Billions of vectors
- Sub-millisecond latency needed
- Memory constraints

DiskANN Overview

- ▶ **Graph-Based ANNS System**
- ▶ Developed by Microsoft Research
- ▶ Scales to billion-point datasets
- ▶ Operates under limited RAM with SSD storage
- ▶ **Vamana Algorithm**
- ▶ Directed graph with bounded out-degree R
- ▶ Greedy beam search for queries
- ▶ α -RNG (Robust) pruning for edge selection
- ▶ **Key Metrics**
- ▶ High recall @ low latency
- ▶ < 5ms latency at 95%+ recall
- ▶ Hope count



Key Limitation of DiskANN

✗ Static Graph

Built once, never changes. Cannot adapt to shifting query patterns.

✎ Distance-Only

Edge selection relies solely on Euclidean distance heuristics.

🧱 Heuristic Pruning

α -RNG pruning doesn't optimize any explicit objective function.

🐶 No Query Learning

Graph ignores feedback from real-world query distributions.

⚡ "The graph does not evolve — it is frozen at index build time"

Our Core Idea

Static Map

Printed once, fixed forever

No real-time updates

Works only for known routes

✗ DiskANN today



Google Maps

Updates routes in real-time

Learns from traffic patterns

Adapts to new conditions

✓ Our Goal

Search



Learn



Refine



Better Search

Our Approach — Three Modules

1

Navigation-Optimization
In greedy search

Edge usefulness scores bias traversal → fewer hops, faster convergence

2

Entropy based
Edge pruning

Diversity-aware pruning replaces pure distance heuristics → richer, better-connected graph

3

Adaptive Graph
Learning

Periodic self-refinement based on query feedback → graph improves over time

Navigation Optimization

in `greedy_search()`

Adaptive Edge Scoring for DiskANN

GraphANN / DiskANN
Vamana Index

Context : What is greedy_search() ?

How it works

- Start at a fixed node in the graph
- Always expand the closest unvisited candidate
- Explore neighbors, keep best L candidates
- Repeat until no better candidates remain
- Return top-K results

The Fundamental Problem

- Purely distance-driven — no intelligence
- No memory of past queries
- Every query starts from scratch
- Query 1 and Query 10,000 take identical paths
- Cannot improve over time

Everything we improve lives inside this one function.

Limitation 1 : Candidate Set Was a Tree

What was the Original used :

`std::set<Candidate>` → red-black tree for candidates
`std::set<uint32_t>` → another tree to track expanded nodes

Why this is slow:

- Every insert = heap allocation + scattered pointers
- CPU cache miss on every node access
- Two trees maintained per query

FIX : Replaced with a flat sorted (`std::vector`)

- New neighbors are batched, sorted, merged with `std::inplace_merge` — one merge per hop
- expanded flag inside entry → 2nd tree gone
- Finding the best node = one forward linear scan over cache-hot memory

Effect: Same logical behavior, radically better memory access pattern.

Limitation 2: Algorithm Never Learned

Original behavior: Every query starts from scratch. The algorithm has no memory of past queries. Query 1 and Query 10,000 explore the same paths in the same order even if certain edges are always productive and others are always dead ends

What was implemented — Edge Usefulness Scoring:

$\text{edge_traversals_}[u \rightarrow v]$ = how many times this edge was followed

$\text{edge_helpful_}[u \rightarrow v]$ = how many times v ended up in the final Top-L result

$\text{usefulness}(u \rightarrow v) = \text{helpful_traversals} / \text{total_traversals}$ [0.0 to 1.0]

- Score 0.0 $\rightarrow$ edge never helped
- Score 1.0 $\rightarrow$ edge always reached a true neighbor
- Cold start $\rightarrow$ all scores = 0.0 $\rightarrow$ identical to original

Effect: Algorithm builds memory of productive paths with every query.

Limitation 3 : Learning Signal Too Sparse

Original attempt :

Only the top-K = 10 nodes were marked as "helpful" after each query.

FIX : Mark all top-L candidates as helpful

```
// Mark ALL top-L candidates as helpful
for each candidate in the final beam (up to L=200 nodes):
    helpful_set.insert(candidate.id)

// Then update edge scores:
for each (u, v) in traversed_edges:
    edge_traversals_[u→v]++
    if v in helpful_set:
        edge_helpful_[u→v]++
```

Intuition: More supervision per query -- faster learning of useful edges .

Effect: Learning signal is 10–20× richer. Edge scores become meaningful much faster — after fewer queries — because more edges get credited on each query.

Entropy-Based Edge Pruning

► Problem

- α -RNG pruning is **distance-based**
- Can remove **useful long-range edges**
- Leads to **local redundancy**

Uses fixed α :

$$d(p, c) > \alpha \cdot d(c, n)$$

Key Idea : Diversity Aware Pruning

Diversity Score

$$\text{diversity}(c, S) = \min_{n \in S} d(c, n)$$

Normalized Diversity

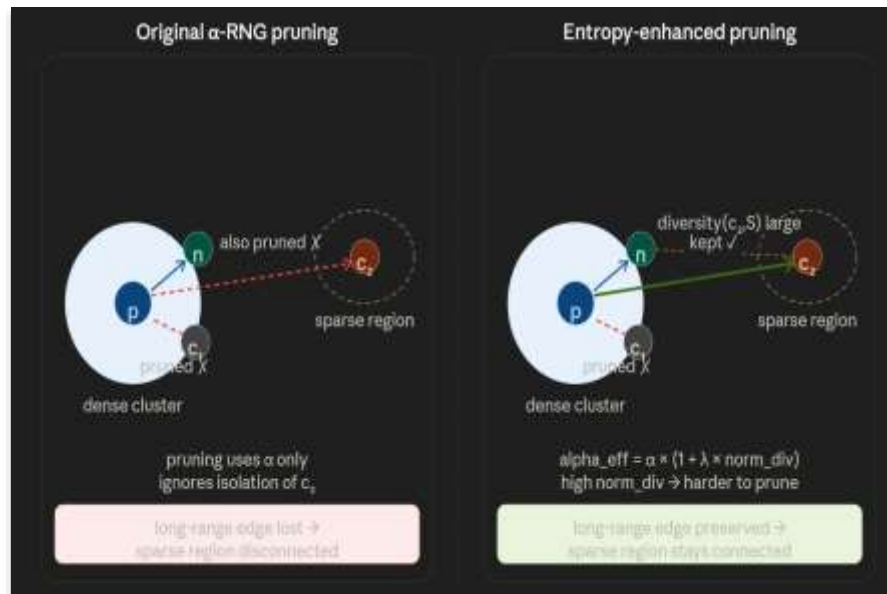
$$\frac{\text{diversity}}{d(p, c)}$$

Dynamic α

$$\alpha_{\text{eff}} = \alpha(1 + \lambda \cdot \text{norm_div})$$

► Impact

- Diverse edges are less likely to be pruned
- Preserves long-range connections
- $\lambda = 0 \rightarrow$ original α -RNG behavior



Adaptive Graph Learning

Our Primary Contribution — Self-Refinement Engine

Step 1: Query Execution

Record traversed edges
Mark helpful edges
Build usefulness signal



Step 2: Adaptive Thresholding

T_remove = 30th percentile
T_boost = 80th percentile
Per-node computation



Step 3: Graph Refinement

Remove weak edges ($<T_{\text{remove}}$)
Boost strong edges ($>T_{\text{boost}}$)
Sort neighbors by usefulness

Search



Update Scores



`refine_graph()`



Improved
Search

🔧 `refine_graph()` triggered every 1,000 queries — incremental, low-overhead, batched updates

What This Achieves

 Fewer Hops	 Lower Latency	 Better Recall	 Improves Over Time
Graph learns shortcuts → less exploration per query	Efficient paths reduce search time significantly	Strong edges kept; weak ones pruned → better results	Self-improving loop → performance gains accumulate

System Comparison

Property	Original DiskANN	Our System
----------	------------------	------------

FINAL RESULTS

L	Recall@10	Avg Dist Cmps	Avg Latency (μs)	P99 Latency (μs)
10	0.7747	643.0	789.1	4643.9
20	0.8903	884.2	1118.8	10132.5
30	0.9334	1101.9	1136.7	2255.2
50	0.9668	1511.4	1583.9	2874.4
75	0.9822	1988.9	2724.7	23268.7
100	0.9891	2438.1	2883.6	5061.7
150	0.9939	3273.3	4143.8	6406.8
200	0.9961	4048.1	5933.7	11753.3

Recall Comparison (Baseline vs Final)

L	Baseline Recall@10	Final Recall@10	Difference
10	0.7760	0.7747	-0.0013
20	0.8896	0.8903	+0.0007
50	0.9662	0.9668	+0.0006
100	0.9890	0.9891	+0.0001

Conclusion

- ✦ Transformed static DiskANN into a dynamic, adaptive system
- ✦ Introduced entropy-based pruning for graph diversity
- ✦ Built navigation-aware search with edge usefulness scoring
- ✦ Designed periodic graph self-refinement (`refine_graph`)
- ✦ **Result: a graph that improves with every query**

★ "ANN systems that improve with usage — not just at build time"